

DevOps + AIOps Take-Home Test

Scope: CI/CD, observability, and operational readiness

Time limit: You have **2 weeks** from receiving this assignment to submit your solution.

Setup

```
make up           # Start stack
make eval         # Run eval suite
make down         # Stop stack
```

Endpoint	URL
Agent API	http://localhost:8080
Prometheus	http://localhost:9090
Grafana	http://localhost:3000 (admin/admin)

Before starting: Run the stack for at least 5 minutes and explore. Understand what the agent does, what metrics exist, and how traffic flows through the system.

Task 1: CI/CD Pipeline (25%)

File: `.github/workflows/ci.yml`

Implement a CI/CD pipeline that ensures only quality code reaches production.

Requirements: - The pipeline should prevent broken builds from being merged
- Deployments should be traceable to specific commits - The eval runner plays a role in quality gates

Design decisions are yours. Justify non-obvious choices in code comments.

Task 2: Alerting Strategy (20%)

File: `prometheus/alert-rules.yml`

The current alerting is incomplete. Design and implement alerts that would be useful for an on-call engineer.

Consider: - What conditions indicate a problem vs. normal operation? - How do you avoid alert fatigue while catching real issues? - What information does an on-call engineer need in the alert?

Run the stack and observe the metrics before deciding on thresholds. Document your reasoning in the alert annotations or comments.

Task 3: Observability Metrics Design (20%)

File: `agent-api/app.py`

The current metrics are minimal. An AI agent system in production needs comprehensive observability.

Your task: 1. Analyze the application and identify what metrics would be valuable for operating this system 2. Implement those metrics in the code 3. Document why each metric matters in a brief comment

Consider what an on-call engineer would need to: - Detect problems before users report them - Debug issues quickly when they occur - Understand system behavior and trends

Study the existing `REQUEST_COUNT` and `REQUEST_LATENCY` patterns.

Task 4: Dashboard Implementation (15%)

File: `grafana/dashboards/agent-monitoring.json`

Several panels are broken (showing “No data”). Fix them using the metrics available (or that you implemented in Task 3).

A good dashboard helps operators quickly understand system health and investigate issues.

Task 5: Incident Response (20%)

Create: `docs/incident-response.md`

You’re the on-call engineer. At 3am, you receive an alert that rejection rate has spiked significantly.

Document your incident response procedure: 1. Initial triage and assessment 2. Investigation steps with specific commands/queries for this system 3. Decision framework for mitigation vs. escalation 4. Post-incident actions

Your document should enable another engineer unfamiliar with this system to handle the incident effectively.

Submission

Submit your solution in **one** of the following ways:

- **Public Git repository** — push your work to your own public GitHub (or other Git hosting) repository and share the link with us.
- **Archive file** — send us a `.zip` or `.tar.gz` archive of the project. Make sure the `.git` folder is included so we can review your commit history.

Deliverables

1. All code changes committed with clear, meaningful commit messages
 2. Working CI pipeline (link to a passing run)
 3. Brief explanation of your alerting threshold choices (can be in comments or a separate file)
-

Evaluation

We evaluate: - **Functionality**: Does it work correctly? - **Judgment**: Are the design decisions reasonable and well-justified? - **Operational thinking**: Would you trust this in production at 3am?

We do not reward verbosity. A concise, well-reasoned solution beats a lengthy generic one.